Finals project Architecture report

רועי אמסלם - 322535436
רביד אללי - 313308504
אורי בלאיש - 207801796
מנחה: אריאל רוט

Abstract

This project is **QDate**, an innovative dating application designed to address the "paradox of choice" in modern digital dating. While traditional platforms maximize swipes and matches, QDate focuses on meaningful decision-making through enforced artificial scarcity. The system operates in two distinct phases: a 14-day learning period with daily matches to gather behavioral data, followed by a curated weekly match phase.

By using machine learning to analyze user intent, messaging patterns, and engagement levels, QDate provides high-quality, intentional connections rather than high-volume, shallow interactions. This approach solves the common frustrations of choice overload for women and low signal-to-noise ratios for men, transforming the dating experience from a game of stimulation into an experiment in adaptive matching and constrained decision-making.

1. Introduction

Finding a meaningful connection in the digital age is challenging. Current dating apps often lead to frustration due to an overwhelming abundance of choices, which actually reduces the perceived value of each match. This project builds **QDate**, a dating application that prioritizes quality over quantity. Unlike traditional platforms that encourage endless swiping, QDate enforces artificial scarcity by providing users with only one match at a time. This constraints-based approach encourages intentionality and reflection, turning the dating process into a more thoughtful and data-driven experience.

2. Problem Statement

Current dating platforms suffer from a critical imbalance and a "paradox of choice" that reduces the quality of outcomes for all users. Men often experience extremely low signal-to-noise ratios and empty interactions, leading to reduced motivation due to intense competition. Conversely, women face a massive overload of options and shallow signals, which often results in decision paralysis or no decision at all.

This abundance of choice creates an environment where the perceived value of an individual connection is diminished. Furthermore, existing apps focus on maximizing swipes rather than learning true user intent. This "intentionality gap" leads to a cycle of frustration and superficial engagement, highlighting the urgent need for a system that uses artificial scarcity to encourage deeper reflection and more meaningful decision-making.

3. Objectives

- **Enforce Artificial Scarcity:** Develop a matching engine that restricts users to exactly one match at a time to increase intention and reflection.

- **Behavioral Data Collection:** Build a system to observe and record passive behavioral data, such as messaging frequency, reply latency, and balance of initiative, rather than relying solely on stated preferences.

- **Two-Phase Matching Lifecycle:** Implement a structured 14-day learning phase followed by a curated weekly matching cycle with built-in cooldown penalties for skipping.

- **Predictive ML Architecture:** Create a roadmap for a machine learning model that evolves from heuristic-based matching to predictive ranking based on conversion likelihood and communication compatibility.

- **Minimalist UX/UI:** Design a calm, non-stimulating interface that emphasizes clear countdowns and progress indicators to reinforce the "slow dating" philosophy.

4. User Stories

1. **As a user**, I want to receive only one match at a time, so that I can focus my full attention on one person without being distracted by other options.
2. **As a user**, I want to see a clear countdown timer for my current match, so that I am encouraged to engage thoughtfully within the 24-hour window.
3. **As a user**, I want the app to provide a minimalist and calm interface, so that I feel less overwhelmed and more intentional during my dating experience.
4. **As a user**, I want to provide feedback after a match expires, so that the system can learn my preferences and improve future suggestions.
5. **As a woman**, I want the system to limit the number of daily matches, so that I can avoid choice overload and decision paralysis.
6. **As a man**, I want to interact in a high-intention environment, so that my efforts aren't lost in a sea of empty interactions and extreme competition.
7. **As a user in Phase 2**, I want to receive one highly curated match per week, so that I can invest more emotional energy into a high-quality connection.
8. **As a developer**, I want to track passive behavioral data like reply latency and messaging frequency, so that the matching algorithm can learn true user intent beyond stated preferences.
9. **As a user**, I want to be notified of a "cooldown penalty" if I skip a curated match, so that I am reminded of the value and intentionality required by the platform.
10. **As a user**, I want to see progress indicators during the 14-day training period, so that I understand how the system is learning about my behavior.

5. Sample Test Code for User Stories

User Story 1: Constrained Daily Match

```
# Test: User receives exactly one match during the 14-day learning period
import requests

# Start a new day for the user
response = requests.post("http://localhost:5000/api/match/daily_generate",
                         json={"user_id": 123})
match_data = response.json()

# [cite_start]Assert that only one match is provided and it has a 24h expiration
assert response.status_code == 200
assert "match_id" in match_data
assert match_data["expires_in_hours"] == 24
print("✓ Daily match constrained to a single instance successfully")
```

 # User Story 2: Behavioral Data Tracking (Passive Observation)

```
# [cite_start]Test: System records messaging latency and frequency [cite: 16, 22]
import requests
import json

payload = {
    "match_id": 456,
    "sender_id": 123,
    "message_length": 45,
    "response_time_seconds": 1200  # Passive data collection
}

res = requests.post("http://localhost:5000/api/analytics/message_event", json=pay
data = res.json()

assert res.status_code == 200
assert "intent_score_updated" in data
print("✓ Behavioral data (latency/frequency) captured successfully")
```

 # User Story 3: Weekly Curated Match (Phase 2)

```
# [cite_start]Test: Weekly match generation with cooldown penalty check [cite: 18
import requests

summary = requests.get("http://localhost:5000/api/match/weekly_curated/123").jso

# Assert the match is framed as high-value and checks for prior penalties
assert "curated_match_id" in summary
assert summary["is_intentional_pairing"] == True
assert summary["cooldown_active"] == False
print("✓ Weekly curated match retrieved with intent validation")
```

 # User Story 4: Post-Match Feedback & Learning

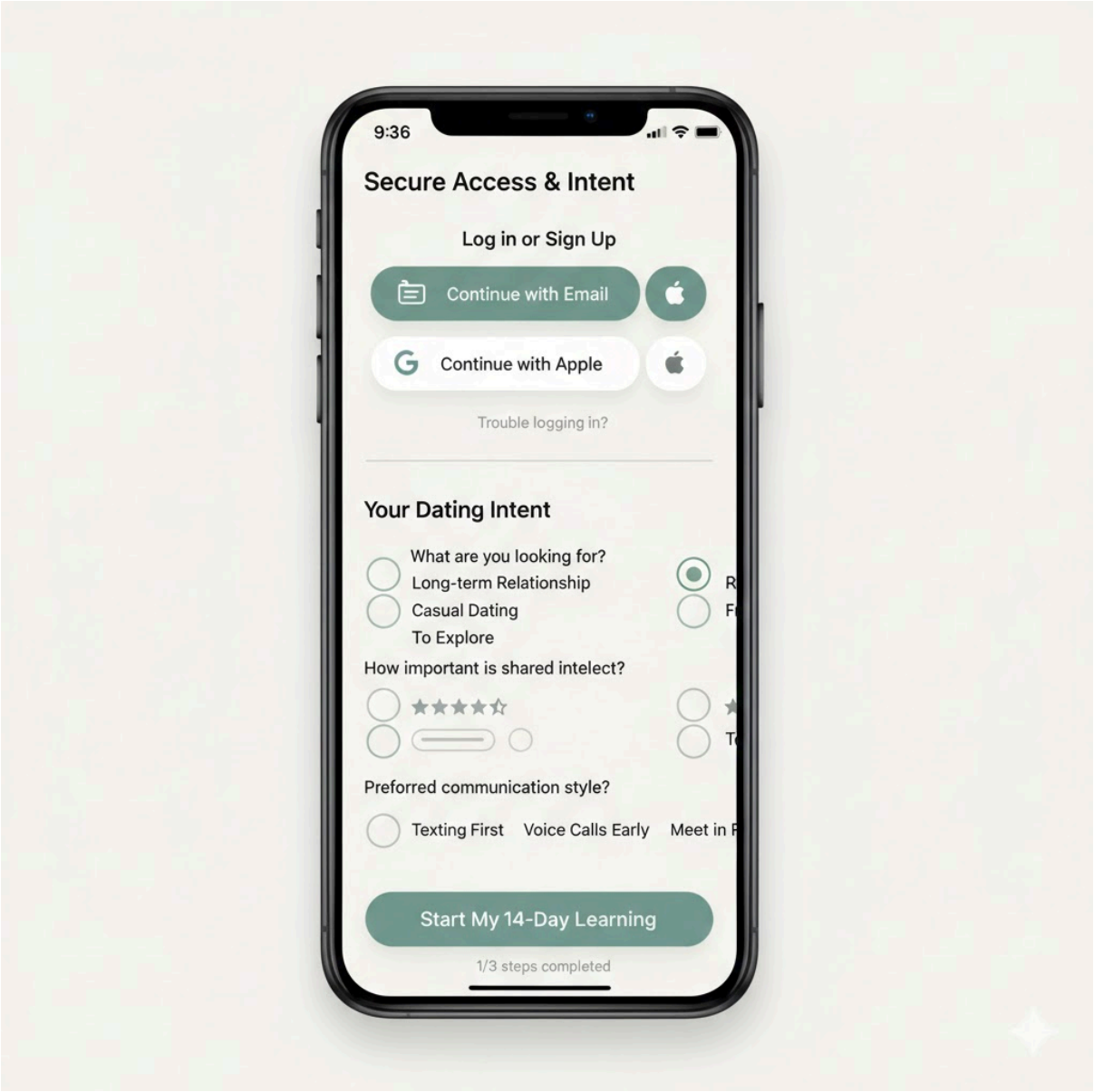
```
# [cite_start]Test: Lightweight feedback collection after match expiration [cite:
import requests

feedback_payload = {
    "match_id": 456,
    "user_id": 123,
    "willingness_to_meet": 4, # Scale 1-5
    "communication_compatibility": 5
}

res = requests.post("http://localhost:5000/api/learning/feedback", json=feedback_
```

5. Application Screenshots (Generate with AI)

Screenshot 1: Login & Intent Onboarding

The login screen reflects QDate's minimalist philosophy. Instead of high-stimulation visuals, it uses a calm aesthetic to set the tone for intentional dating.

- **Secure Access:** Standard login via email or OAuth (Google/Apple).
- **Initial Intent Setup:** A brief series of questions to establish the user's baseline preferences before entering the Phase 1 training period.



Screenshot 2: The Daily Focus (Phase 1)

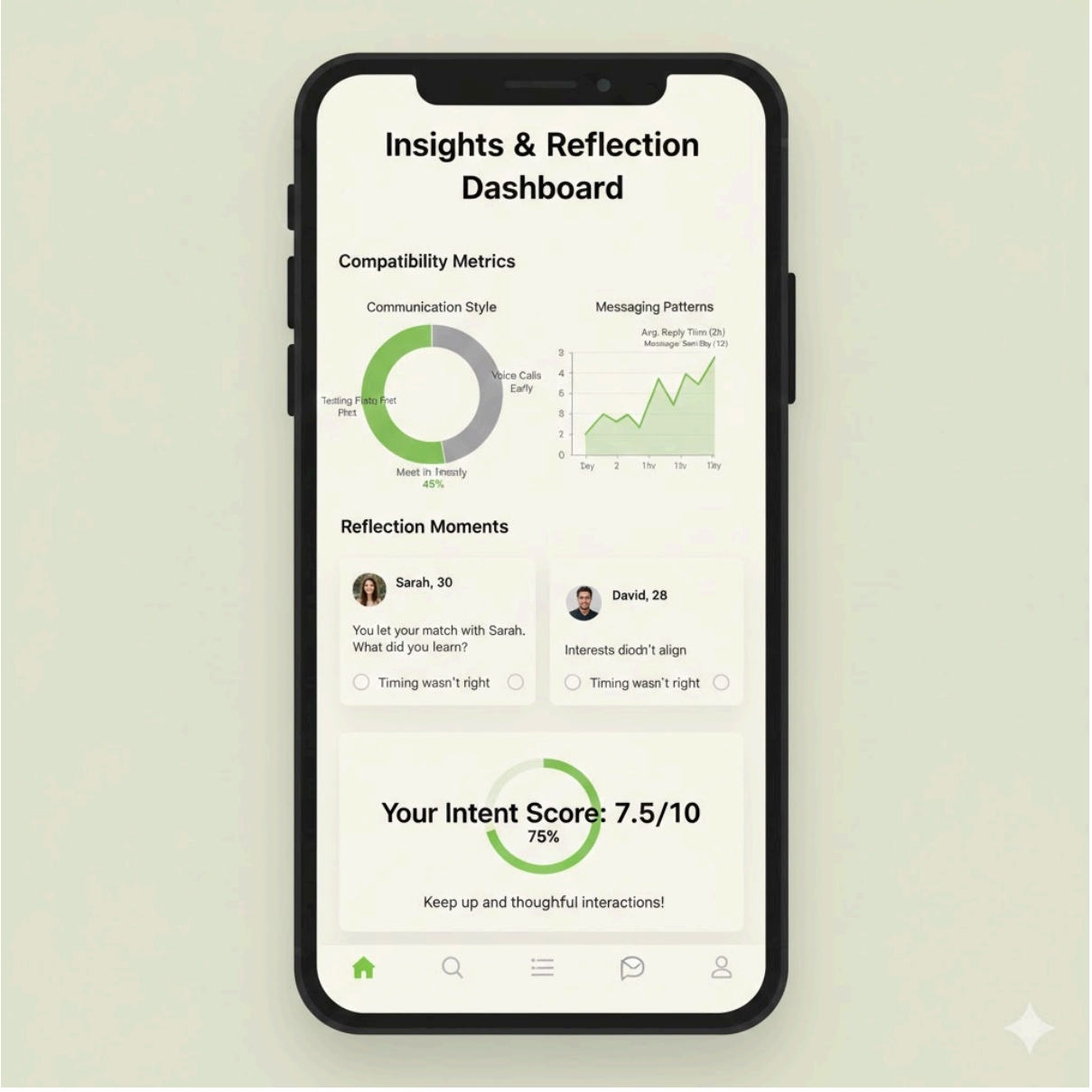
This is the primary interface during the first 14 days, where choice is strictly constrained to increase reflection.

- **Single Profile View:** Only one potential match is displayed, emphasizing depth over breadth.
- **Scarcity Indicators:** A prominent 24-hour countdown clock encourages active engagement and timely decision-making.
- **Behavioral Progress:** A small indicator showing how many days remain in the "Learning Period" before graduating to curated matches.

Screenshot 3: Insights & Reflection Dashboard

Replacing traditional "swipe history," this dashboard shows the user how the system is learning their intent.

- **Compatibility Metrics:** Visual graphs showing communication compatibility and messaging patterns observed by the ML model.
- **Reflection Moments:** Prompts asking for lightweight feedback on expired matches to refine the predictive ranking model.
- **Intent Score:** A summary of the user's "intentionality level," rewarding high-quality interactions rather than match volume.



6. Typical User Flows

1. User Flow for Engagement During Phase 1 (Learning Period)

User receives a push notification that their match of the day is ready

→ Opens the **QDate** app

→ Logs in securely

→ Views the single profile of the day, emphasizing scarcity over stimulation

→ Initiates a conversation within the 24-hour time-boxed window

→ System passively observes messaging frequency and reply latency

→ 24-hour window expires and the match is removed

→ User provides lightweight feedback on the interaction to help the system learn intent

2. User Flow for Phase 2 (Curated Weekly Match)

User receives a notification for their highly curated weekly match

→ Opens the **QDate** app

→ Navigates to the "Weekly Focus"

→ Reviews a deeply curated profile based on behavioral data from Phase 1

→ Decides to "Accept" (starting a high-investment interaction) or "Skip"

→ If "Skip" is chosen, the user views a notification about the cooldown penalty to reinforce intentionality

→ User views their **Intent Dashboard** to see progress indicators and compatibility insights learned by the ML model

7. Implementation

8.1 Technologies Used

Frontend (Client Application)

- **React Native (TypeScript)** – Cross-platform mobile development (iOS & Android).
- **Native Modules (Kotlin & Swift)** – For accessing device-level telemetry and high-precision interaction tracking.
- **Dual-Stream Navigation Logic** – Custom state management to handle the "Daily Focus" (real match) and "Calibration" (fake profile training) simultaneously.
- **Lottie & Tailwind CSS** – Used to maintain a calm, minimalist aesthetic that reduces decision fatigue and encourages intentionality.

Behavioral Analysis & Processing

- **Interaction Telemetry Engine** – Captures granular data during both real and calibration sessions:
    - **Dwell Time Tracking:** Measures time spent on specific profile elements (bio, attributes, photos).
    - **Interaction Latency:** Records the time delta between profile presentation and user action.
- **Impulsivity Monitor** – Heuristic engine that analyzes rejection velocity. If "fast-paced rejection" is detected, it triggers cooling-off periods or reflection prompts to ensure a fair chance is given to each match.
- **Local Storage (SQLite / AsyncStorage)** – Stores the 14-day training state, local intent vector progress, and the 24-hour countdown timer.

Cloud Backend

- **Firebase Authentication** – Secure login via email or OAuth (Google/Apple) to maintain a frictionless onboarding experience.
- **Node.js & Express (API)** – Serves as the primary gateway for serving the daily real match and processing calibration data.
- **Supervised Learning Service (Python)** – Analyzes interactions with "Scale Profiles" (known fake profiles) to explicitly map user behavior to known personality traits.
- **Firebase Cloud Messaging (FCM)** – Dispatches the daily notification when the new focused match is ready, avoiding the high-frequency "noise" of traditional apps.

Database & Intelligence (Cloud)

- **PostgreSQL with pgvector** – Relational database used to store:
    - **User Intent Vectors:** High-dimensional arrays representing learned preferences.
    - **Vector Similarity Search:** Uses Cosine Similarity to identify compatible matches within a user's cluster.
- **K-Means Clustering Service** – Periodically splits the user base into "cliques" where tastes and behavioral patterns are similar.
- **Calibration Library** – A curated set of "Scale Profiles" with pre-defined labels (e.g., highly creative, highly logical) used to test and refine the user's intent vector.